

Abenix

The Edge Agent Guide

How AI agents in the cloud and on the plant floor cooperate over MQTT — for clients, plant operators, and application developers.

An Abenix client-facing reference document.

Contents

The Abenix Edge Agent Guide

Part I — The big picture

Part II — Agent fundamentals

Part III — The MQTT bus, explained gently

Part IV — How edge and cloud work in tandem

Part V — Real-life scenarios

Part VI — How an application uses Abenix

Part VII — Anatomy of one execution

Part VIII — Security and governance

FAQ

Glossary

The Abenix Edge Agent Guide

A walkthrough for clients, plant operators, and application developers. No prior knowledge of AI agents, MQTT, or Kubernetes assumed.

Who this is for

This guide is for three kinds of readers, all of whom will get something out of every chapter:

- 1 **The plant operator or business sponsor** — you want to understand *what* an Abenix edge agent is, *why* it exists, and *what it does for your operations*. Skip the Python and YAML — the diagrams and the real-life scenarios in Part V are written for you.
 - 2 **The application developer** — you want to know *how* to call Abenix from your own software, *how* an agent is built, and *how* the cloud and edge halves talk. Read everything; the SDK chapter (Part VI) is your home.
 - 3 **The plant IT / OT lead** — you want to know *how* the bytes move, *what* gets installed where, and *how* the security model works. The lifecycle, MQTT, and security chapters (Parts III, IV, VIII) are written for you.
-

Executive summary

Abenix is a platform that lets you build AI agents — small specialised programs that read information, reason about it, and take action — and run them either in the cloud or right next to your equipment.

The cloud half handles the heavy thinking: long pipelines, big knowledge graphs, big language models, multi-step reasoning, multi-human approvals. The edge half handles the urgent local work: a vibration anomaly that needs a verdict in milliseconds, a refrigerated trailer that's lost 4G signal, a SCADA alarm that can't wait for a round-trip to AWS.

The two halves talk through a single message bus called **MQTT** — a lightweight protocol that's been the lingua franca of industrial telemetry for two decades. Agents flagged "edge eligible" are compiled into a tamper-proof bundle, signed by the platform, shipped over MQTT to the gateway pod next to the equipment, and run there. The same agent definition works in both places. You author once, and decide later where it executes.

This guide walks you through the whole picture, from "what is an agent" to "here's a wind farm running on this exact stack today."

Part I — The big picture

What problem does Abenix solve?

Imagine three real situations.

Situation A: A wind turbine's vibration sensor spikes. You have ten seconds to decide if this is a bearing about to fail (shut down, dispatch a crane, prevent a \$2 M gearbox replacement) or a wave hitting the platform (ignore, keep generating). Sending the raw 50 kHz vibration stream to a cloud model is too slow, too expensive, and too fragile when the satellite link is flaky.

Situation B: A pharma shipment of vaccines drops below 2 °C for nine minutes. GDP regulations require an excursion adjudication: was this a real loss, was the cooling system at fault, who pays? The decision needs the policy KB, the SOP knowledge graph, the prior-incident history, and three humans signing off in a TTL-bound window. A small Python script on the truck cannot do this — it needs the platform.

Situation C: A field technician at a remote substation needs an OEM-cited repair procedure for a fault code she's seeing. She has spotty 4G. She speaks the symptom. She uploads a photo of the damage. She wants the procedure, the safety gate, and a populated work order — and the cloud may or may not be reachable.

These three situations share one feature: **the right answer needs reasoning, not just retrieval.** They differ in where the reasoning belongs. Situation A wants the model on the gateway. Situation B wants the model in the cloud with a human gate. Situation C wants both — partial work on the device, the rest in the cloud when the link comes back.

Abenix is the platform that says: *write the reasoning once, decide at deploy time where it runs, and let the two halves cooperate over a common bus.*

Cloud agents vs edge agents — a one-sentence intuition

Think of a cloud agent as a **senior consultant in head office** with access to the entire knowledge base, the email system, and the corporate finance tools. They take on slow, complex, multi-step problems. They can call other consultants. They can ask for human signoff.

Think of an edge agent as a **field engineer with a toolbox at the site.** They see the equipment directly. They make fast local decisions on a small set of well-rehearsed problems. They report up to head office. They can flag something for the senior consultant to look at later.

The two work together every day. The senior consultant tells the field engineer what to focus on this week. The field engineer escalates anything outside the rehearsed playbook. They use a common radio channel — that radio channel is MQTT.

Why MQTT?

MQTT (Message Queuing Telemetry Transport) is a publish-subscribe message protocol invented in 1999 for satellite-linked oil pipelines. It's three things at once:

- A **postal system** — anyone can drop a message on a topic (publish), anyone listening to that topic gets it (subscribe).
- **Bandwidth-efficient** — a small fixed-size header, typically 2–10 bytes of overhead per message. Designed for satellite, GSM, and dial-up.
- **Stable under flaky networks** — messages can be retained, queued, replayed, and delivered with three quality levels.

Most plants already have an MQTT broker running. Most SCADA, DCS, and historian systems already speak MQTT or can speak it through a gateway. By using MQTT as the cloud <-> edge bus, Abenix slots into infrastructure that's already there — no new firewall holes, no new vendor relationships, no new protocol to teach the OT team.

Part II — Agent fundamentals

What is an agent?

An **agent** in Abenix is a small specialised program made of three things:

- 1 **A name and a description** — `iot-pump-edge-classifier`, "you receive a vibration window, you return a severity verdict."
- 2 **A model and a system prompt** — which language model to use (Claude Haiku, GPT-4o, Gemini Flash) and the standing instructions that shape every conversation.
- 3 **A list of tools** — the agent's hands. Things like `mqtt_publish` (send a message to a topic), `tsdb_query` (read time-series data), `connector_call` (talk to SAP), `code_executor` (run a snippet of Python).

When an agent runs, it gets some **input** (a JSON blob, a file, a text prompt). It reasons over the input using the model. If it needs to act on the world — read a database, send an email, call a SCADA system — it calls a tool. The tool's output goes back into the conversation. The agent reasons again. Loop until the agent decides it's done, then return a final structured result.

That loop is the only thing an agent ever does. Different problems need different combinations of tools, prompts, and models — but the loop is universal.

What is a tool?

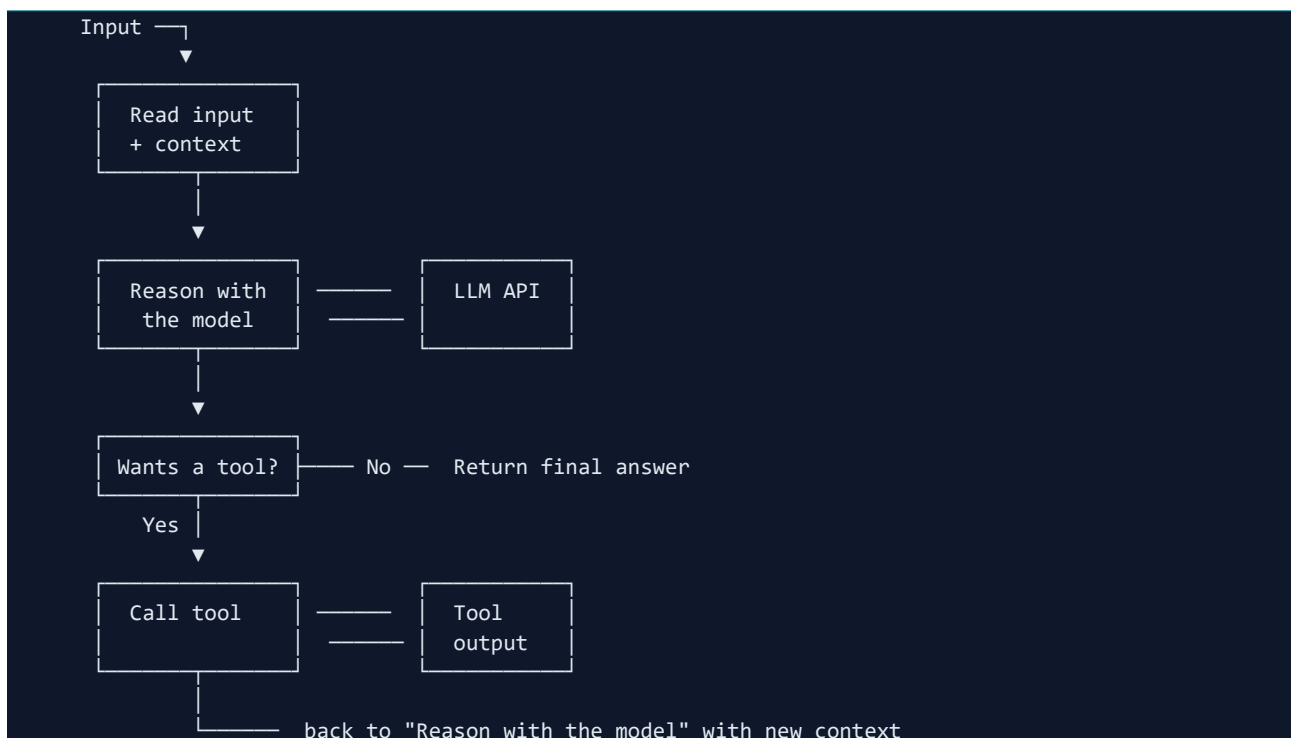
A tool is a small named function the agent can call. Tools are how agents interact with the real world. Without tools, an agent is just a chatbot that returns text. With tools, an agent can read sensors, write to PLCs, file work orders, call humans, and trigger other agents.

Abenix ships 100+ tools out of the box across nine families:

Family	Examples
Web	<code>web_search</code> , <code>web_scrape</code> , <code>structured_extract</code>
Knowledge	<code>kb_search</code> , <code>kb_ingest</code> , <code>atlas_traverse</code> (graph walk)
Code	<code>code_executor</code> (sandboxed Python/Node/Go/Rust/Java/Ruby)
Data	<code>tsdb_query</code> , <code>postgres</code> , <code>s3</code> , <code>csv_reader</code> , <code>parquet_reader</code>
Comms	<code>slack</code> , <code>email_sender</code> , <code>webhook</code>
Productivity	<code>linear</code> , <code>jira</code> , <code>notion</code> , <code>github</code>
Vision + audio	<code>image_analyzer</code> , <code>audio_stt</code>
Industrial	<code>mqtt_publish</code> , <code>mqtt_subscribe</code> , <code>opcua_write</code> , <code>connector_call</code>
Governance	<code>approval_gate</code> , <code>windowed_state</code> , <code>subscribed_feed</code>

A platform admin curates which tools each agent can use. An agent that needs to file work orders gets `connector_call` to ServiceNow. An agent that needs sub-second alarm triage gets `mqtt_publish` and `windowed_state`.

How does an agent run?



That's it. Every agent in Abenix — cloud or edge — runs that loop. The differences between cloud and edge are only:

- **Which tools the agent is allowed to call** (the edge has a tighter whitelist).
- **Where the agent's process lives** (a pod in the cluster vs a pod on the gateway).
- **Which model the agent talks to** (the edge usually picks a smaller, faster model — Haiku rather than Sonnet — because the network may be flaky).

A cloud agent example — [claimsq-adjudicate](#)

Imagine a customer files a car insurance claim. The platform receives the FNOL (First Notice of Loss): a description, a photo of the damage, the policy number. A 6-stage cloud pipeline kicks off:

- 1 **FNOL Intake agent** — parses the free text into structured fields (vehicle, location, third party, severity).
- 2 **Policy Match agent** — uses [kb_search](#) to find the exact clauses in the policy KB that govern this claim type.
- 3 **Damage Assess agent** — uses [image_analyzer](#) (a multimodal model) to estimate severity from the photo.
- 4 **Fraud Screen agent** — uses [windowed_state](#) to check for repeat-claim patterns from this customer.
- 5 **Valuator agent** — uses [connector_call](#) against the parts-pricing service to compute the dollar amount.
- 6 **Claim Decider agent** — synthesises everything and proposes approve / partial / deny with cited clauses.

This entire pipeline runs in the cloud because it needs the full policy KB (~50 MB), the multimodal vision model, and the parts-pricing connector with its TLS-protected credentials. It's slow (8–15 seconds per claim) but auditable — every clause cited, every photo measurement logged, every connector call audit-trailed.

An edge agent example — [iot-pump-edge-classifier](#)

Same platform. Different problem. A sensor on a wind-turbine pump publishes a vibration window every second to MQTT topic [pump/vibration.raw](#). We want a verdict (low / medium / high severity) within 50 milliseconds, and we don't want the verdict to depend on the satellite link to the cloud being up.

The edge agent:

- **System prompt:** "You receive a JSON `{samples: [float], sample_rate_hz: int}`. Use the embedded Python tool to compute RMS and the dominant FFT frequency. Return ONLY `{severity, peak_hz, rms_g}` based on these rules: `rms>0.5` OR peak between 95–105 Hz = high, `rms>0.3` = medium, else = low."
- **Model:** Claude Haiku (small, fast, ~150 ms cloud round-trip).
- **Tools:** `code_executor` (to run the FFT), `mqtt_publish` (to write the verdict back to `plant/pump/severity`).
- **Edge constraints:** max payload 8 KB, max runtime 5 seconds, only allowed to publish to `plant/pump/severity`.

This agent runs on a small pod sitting on the plant LAN, next to the SCADA system. The pod was helm-installed once, registered itself with the platform, and now hot-loads new versions of the agent whenever the platform pushes one over MQTT.

The two agents — cloud `claimsiaq-adjudicate` and edge `iot-pump-edge-classifier` — never meet directly. But they live in the same catalogue, are written in the same YAML format, and use the same SDK pattern. That's the point.

Part III — The MQTT bus, explained gently

The post-office analogy

Imagine a town with a single huge post office. Anyone in the town can drop a postcard into the post office addressed to a topic — say, `weather/rain` or `mayor/announcements`. Anyone in the town can subscribe to a topic and the post office automatically forwards every postcard on that topic to them.

That's MQTT in one paragraph.

- The **post office** is the **broker** (in Abenix: mosquitto, the most popular open-source MQTT broker — it boots in 30 MB, ships in our docker-compose, ships in our helm chart).
- A **postcard** is a **message** (any bytes — JSON, binary, plain text — typically a few hundred to a few thousand bytes).
- A **mailbox** is a **topic** — but unlike addresses, topics are hierarchical strings: `pump/vibration.raw`, `cold-chain/truck-42/temperature`, `plant/floor-3/alarm/severity`.

Topics — the address scheme

Topics are forward-slash-separated strings, like file paths. They support wildcards on subscribe:

- `pump/vibration.raw` — exactly that topic.
- `pump/+/temperature` — single-level wildcard (`pump/eastern-1/temperature`, `pump/eastern-2/temperature`, etc.).
- `cold-chain/#` — multi-level wildcard (everything under cold-chain).

Good topic hygiene is half the battle. A common pattern: `<site>/<asset-type>/<asset-id>/<measurement>` — `plant-3/pump/p-12/vibration-rms`.

Publish, subscribe, and the three QoS levels

There are exactly two verbs in MQTT: **publish** (drop a message on a topic) and **subscribe** (tell the broker which topics you care about, then receive forwarded messages). Both verbs take a **QoS** — quality of service — which controls delivery guarantees:

QoS	Meaning	When to use
0	"fire and forget" — at most once delivery	High-volume telemetry where loss is OK (every-second sensor reads).
1	"at least once" — guaranteed but possibly duplicated	Most operational messages (alarms, deploy events, status updates).
2	"exactly once" — guaranteed and deduplicated	Rare; only critical settlement-style messages.

Abenix uses QoS 1 for bundle delivery (we want it to land), QoS 0 for high-frequency telemetry, QoS 1 for command-and-control.

Retained messages — the "current state" trick

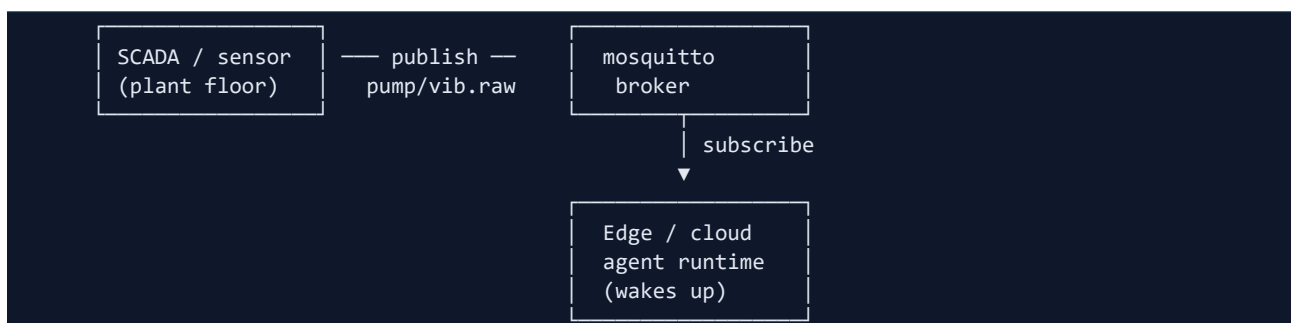
Normally a subscriber that joins late doesn't see old messages — they were already forwarded. But MQTT has a flag called **retained**: when set on a publish, the broker keeps the latest message on that topic and replays it to every new subscriber.

This makes retained-on-publish a clean pattern for "current state" topics. Example: `plant-3/pump/p-12/state` retained = `{"running": true, "rpm": 1480}`. A new dashboard joining the broker sees the current state immediately, no polling.

The four ways Abenix uses MQTT

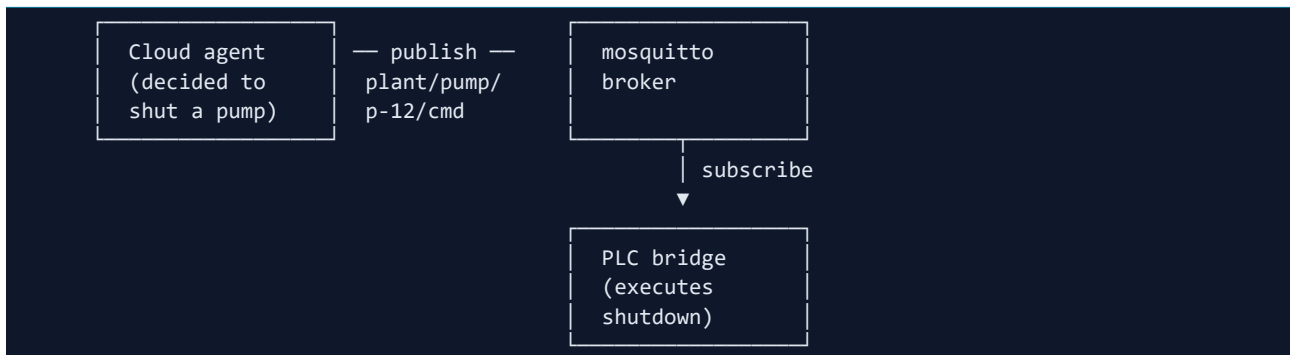
There are four distinct patterns. Understanding them is the key to understanding the whole edge system:

Pattern 1 — Sensor -> Agent (telemetry trigger)



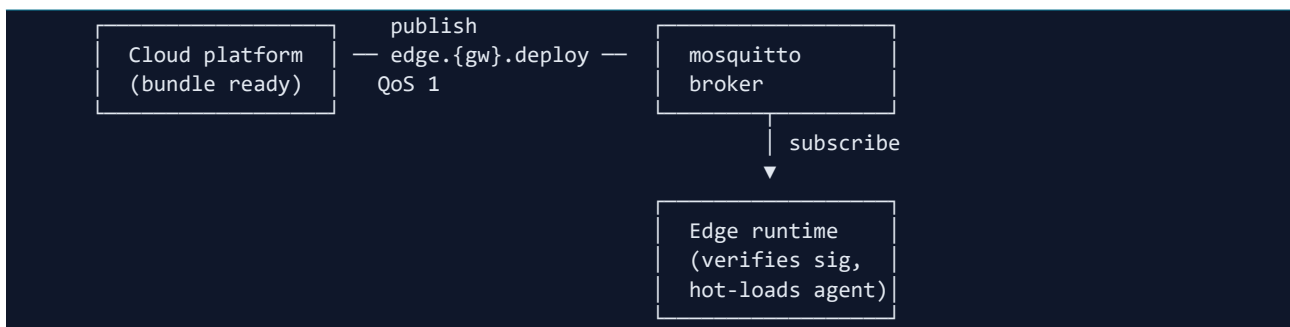
The sensor publishes telemetry on a topic. An agent's runtime is subscribed to that topic. Every new message wakes the agent and feeds the bytes in as input. Zero polling, zero glue code. (Inside the platform: configure a `mqtt_trigger` on the agent and point it at the topic pattern.)

Pattern 2 — Agent -> Equipment (command-and-control)



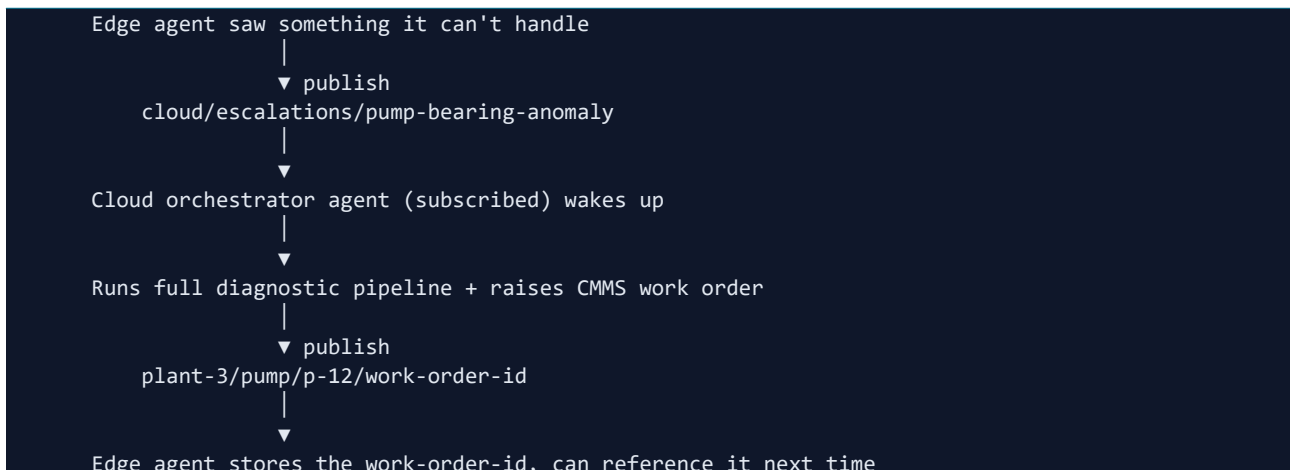
An agent decides on an action — shut a pump, raise a setpoint, open a valve — and publishes a command on a topic the equipment is subscribed to. (Inside the platform: the agent's `mqtt_publish` tool is configured with the topic, payload schema, and a per-agent ACL that restricts which topics the agent can publish to.)

Pattern 3 — Platform -> Edge (signed bundle delivery)



The platform compiles an agent into a `.agent` bundle (a signed tarball), publishes the bytes on the gateway's deploy topic, and the gateway's runtime hot-loads it. This is **how a cloud-authored agent gets onto the edge**.

Pattern 4 — Edge <-> Cloud (handoffs and escalations)

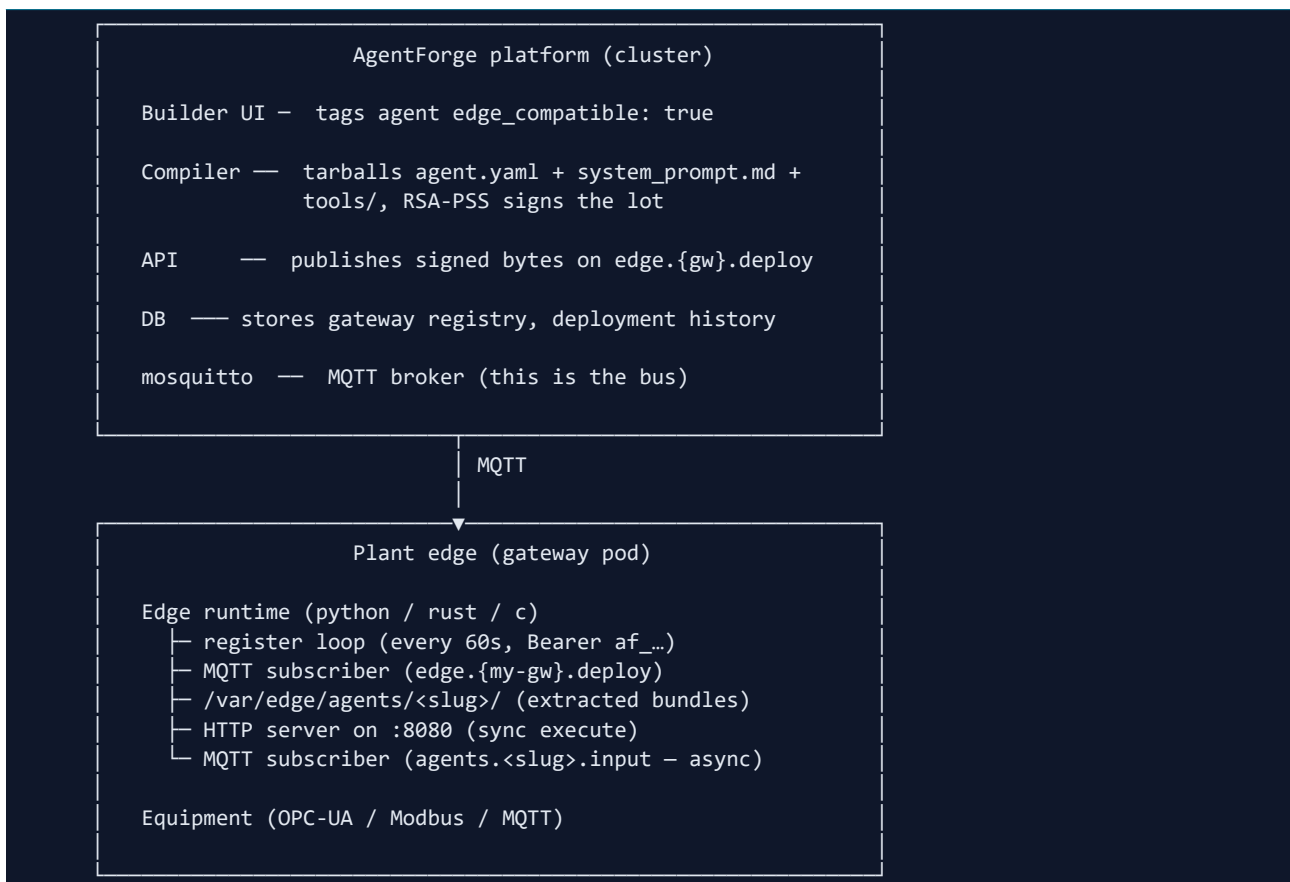


Edge agents handle 90% of cases locally and *escalate* the rest to a cloud agent by publishing on an agreed escalation topic. The cloud agent does the deep work and publishes its result back. This is the real cooperation pattern between cloud and edge.

Part IV — How edge and cloud work in tandem

This part is the heart of the document. We'll walk through the entire lifecycle of an edge agent, from the moment a developer marks an agent as edge-eligible to the moment it's running on a turbine generating verdicts.

The big picture



Now let's walk each step.

Step 1 — Authoring an agent

A developer or domain expert opens the **Builder** in the Abenix web UI. They configure:

- **Name and slug:** `iot-pump-edge-classifier`.
- **Model:** Claude Haiku (small + fast).
- **System prompt:** the one we showed earlier.

- **Tools:** `code_executor`, `mqtt_publish`.
- **Edge eligibility:** ticks a box. The Builder validates the configuration against the edge whitelist — only six tools are allowed, and the agent must declare its `edge_constraints` (max payload, max runtime, allowed MQTT topics).
- **Status:** `draft`. Nothing is deployed yet.

The agent is now in the platform's catalogue. It can be tested in the cloud (via the SDK or the `/agents/{id}/execute` endpoint) before any edge work begins. This matters: **you debug in the cloud, you ship to the edge.**

Step 2 — Provisioning a gateway

Meanwhile, the plant IT team prepares a gateway. Three steps:

- 1 **Mint a registration token.** From the platform UI: **API Keys -> New key**, scopes `agents:execute`, `edge:register`. The `af_...` value goes into the gateway's secret store.
- 2 **Pick a runtime variant.** Python (~80 MB, easy to extend), Rust (~25 MB, single static binary, rugged industrial PCs), or C (~12 MB, ultra-constrained PLCs). The `/edge` page in the platform UI shows three cards with copy-paste install commands.
- 3 **Helm-install** (or `docker run`, on a non-Kubernetes plant gateway):

```
helm install abenix-edge ./infra/helm/edge-runtime \
  -n abenix-edge \
  --set platform.url=https://abenix.your-corp.com \
  --set platform.token=$EDGE_REGISTRATION_TOKEN \
  --set gateway.id=plant-3-gw-1 \
  --set mqtt_url=mqtt://mqtt.your-plant:1883 \
  --set anthropic_api_key=$ANTHROPIC_API_KEY
```

The pod boots, the runtime starts, and within 60 seconds two things have happened:

- The pod has called `POST /api/edge/gateways/register` on the platform, identifying itself with `gateway_id: plant-3-gw-1`. The platform's database now has a row for this gateway.
- The pod has subscribed to MQTT topic `edge.plant-3-gw-1.deploy` (its private deploy channel).

Refresh the platform's `/edge` page and the new gateway appears in the **Registered gateways** grid with `last_seen_at: 12 seconds ago`.

Step 3 — Compiling the bundle

Back to the developer. They open `/edge`, click the gateway card, click **Deploy agent**, pick `iot-pump-edge-classifier` from the catalogue. The platform now does six things:

- 1 **Validate the agent is still edge-eligible** (no tools have been added that violate the whitelist).
- 2 **Resolve the manifest** — pull the YAML, the system prompt, any inline tools from the database.
- 3 **Build the bundle** — an uncompressed tar containing:
 - `agent.yaml` (manifest)
 - `system_prompt.md` (the prompt)
 - `tools/*.py` (any inline tool implementations)
 - optional `model_weights/` (reserved for distilled local models in a future release)
- 1 **Sign it** — compute SHA-256 of the tar minus a placeholder `signature.sig`, then sign the digest with RSA-PSS (MGF1-SHA-256, salt-len 32) using the platform's signing key. Append the signature as the final tar entry.
- 2 **Publish on MQTT** — `mosquitto.publish('edge.plant-3-gw-1.deploy', bundle_bytes, qos=1)`.
- 3 **Record the deployment** — write a row to `edge_gateways.deployed_agents` with the bundle digest and timestamp.

If MQTT publish fails for any reason (broker unreachable, connection refused) the platform falls back to a direct HTTP POST against the gateway's `endpoint_url`. Either way, the bundle reaches the gateway.

Step 4 — Hot-loading on the edge

The edge runtime, which has been subscribed to `edge.plant-3-gw-1.deploy`, receives the bundle bytes. It does:

- 1 **Verify the signature.** It loads the public key from `/etc/edge/signing_pub.pem` (mounted by helm), separates the signature from the rest of the tar, recomputes SHA-256, and runs RSA-PSS verify. **Mismatch -> refuse to load and log the attempted tampering.** This is non-negotiable: an unsigned or wrongly-signed bundle never executes.
- 2 **Re-validate the tool whitelist.** Belt and braces — even though the compiler already enforced this, the runtime checks again. An attacker who somehow forged a bundle declaring `kb_search` or `atlas_*` would still be refused.
- 3 **Extract the tar to** `/var/edge/agents/iot-pump-edge-classifier/`.
- 4 **Register the slug in its in-memory router.** Now `GET /agents` lists `iot-pump-edge-classifier` and the bundle digest. `POST /agents/iot-pump-edge-classifier/execute` is live.
- 5 **Subscribe to** `agents.iot-pump-edge-classifier.input` if the manifest configured async invocation.

Total elapsed time from "click Deploy" to "agent ready": typically 200–800 ms on a fast LAN. The runtime hot-loads — no pod restart, no downtime, no impact on other agents already deployed on the same gateway.

Step 5 — Running the agent

There are two ways to invoke an edge agent.

Sync invocation — direct HTTP

A caller on the plant LAN — typically a SCADA bridge or a small subscriber service — sends a POST:

```
POST http://abenix-edge.plant-3:8080/agents/iot-pump-edge-classifier/execute
Content-Type: application/json
{"samples": [0.012, -0.014, 0.011, ..., -0.020], "sample_rate_hz": 4096}
```

The runtime kicks off the agent loop. The agent reasons, calls `code_executor` to run an embedded Python snippet that computes RMS and the FFT peak, gets the numbers back, applies the rule, and returns:

```
{
  "slug": "iot-pump-edge-classifier",
  "duration_ms": 47,
  "result": {
    "severity": "high",
    "peak_hz": 102.3,
    "rms_g": 0.61
  }
}
```

If `severity == high`, the agent's last move is to call `mqtt_publish` and write to `plant-3/pump/p-12/severity` (a topic the agent is permitted to publish to per its ACL). The plant's alarm desk dashboard, subscribed to that topic, lights up red.

Async invocation — MQTT trigger

For high-volume telemetry, sync HTTP is overkill. Configure the agent's manifest with `triggers: [{type: mqtt, topic: pump/vibration.raw}]`, redeploy the bundle, and the runtime subscribes. Now every vibration packet on the plant's `pump/vibration.raw` topic feeds the agent automatically. The agent runs, publishes its verdict on `plant-3/pump/p-12/severity`, and goes back to listening. No HTTP, no glue.

Step 6 — Day-2 operations

The agent is running. What happens over time?

- **The platform monitors the gateway.** Every 60 seconds the runtime re-registers; if it misses a check-in for >5 minutes, the platform's dashboard flips the gateway state to [stale](#) and an alert fires.
- **Updates ship over the same channel.** The developer changes the system prompt or swaps in a smaller model — clicks Deploy again — the platform pushes a new bundle on the same topic, the runtime hot-loads it. No downtime.
- **Audit trail is unified.** Every edge execution emits a structured log line that flows back to the platform's observability stack. Cloud and edge executions sit in the same [/executions](#) page, the same Grafana dashboards, the same [/alerts](#) failure-code rollups.

That is the complete edge lifecycle.

Part V — Real-life scenarios

Five end-to-end stories. Each one is built on top of patterns and components that ship in this repository — they're not hypothetical.

Scenario 1 — Wind turbine vibration monitoring

The plant. A 50-turbine offshore wind farm in the North Sea. Each turbine has accelerometers on the gearbox, bearings, and blade roots. The vibration data is published continuously to a local MQTT broker on the platform jack-up at 4 kHz. The cloud link is a satellite uplink with 800–1200 ms of latency and frequent dropouts during storms.

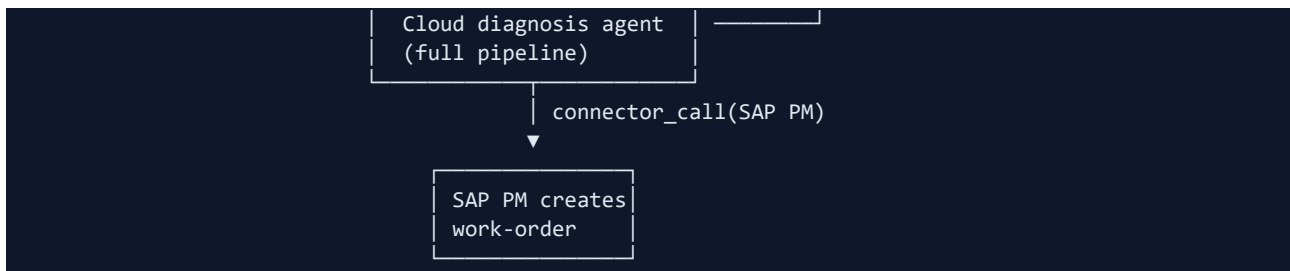
The problem. A bearing failure that goes unspotted for 24 hours can grow into a gearbox replacement — 2–3 weeks of lost generation, \$2 M parts + crane bill. A bearing failure caught at the early-warning stage is a \$4k swap and one shift of downtime.

The agents.

- [iot-pump-edge-classifier](#) (edge, runs on a Rust runtime on the platform jack-up). System prompt: classify a 1-second vibration window into low / medium / high severity using FFT peak and RMS. Tools: [code_executor](#), [mqtt_publish](#).
- [iot-pump-diagnosis](#) (cloud, runs in the AKS cluster). System prompt: given a window of high-severity vibration plus the asset's last 30 days of history (from the TimescaleDB hypertable), classify the failure mode (bearing / imbalance / misalignment / cavitation) and propose a maintenance action. Tools: [tsdb_query](#), [kb_search](#) against [industrial-iot-knowledge](#), [connector_call](#) to the SAP PM CMMS.

The flow.





Wall-clock timeline of one bearing-failure event:

- T+0 ms — Vibration sensor publishes a 4096-sample window to `pump/vibration.raw`.
- T+12 ms — Edge runtime receives, forwards into the classifier agent.
- T+47 ms — Classifier returns `severity: high, peak_hz: 102.3, rms_g: 0.61`. Publishes to `plant-3/pump/p-12/severity` and `cloud/escalation/bearing-anomaly` (with the original window attached).
- T+50 ms — Alarm desk light flips to amber.
- T+1.4 s — Cloud runtime, subscribed to `cloud/escalation/#`, picks up the message. Wakes the diagnosis agent.
- T+8 s — Diagnosis agent has pulled the last 30 days of vibration RMS from TimescaleDB, used `kb_search` to find the matching SOP for "bearing degradation, ISO 10816 zone C", and asked the SAP PM connector to draft a work order.
- T+9 s — Work order ID `WO-2026-04-1893` published back on `plant-3/pump/p-12/work-order-id`.
- T+9.1 s — Edge agent stores the WO ID in its `windowed_state` so the next 60 seconds of high-severity readings reference the same WO instead of opening duplicates.

What this gives the operator. Sub-second local detection, multi-second cloud diagnosis with full historical context, and an automatically drafted work order — all without humans touching a keyboard.

What this gives the auditor. Every step is logged. The edge execution is in `/executions` with the bundle digest. The cloud pipeline is in `/executions` with all six agent outputs and the connector call. The work order has the audit ID embedded in its description.

Scenario 2 — Pharma cold-chain excursion

The plant. A pharmaceutical distributor moving vaccines from Frankfurt to Riyadh. The shipment is in a refrigerated container with a Sensitech logger publishing temperature every 5 minutes over GSM to the MQTT broker.

The problem. Vaccines are required by GDP (Good Distribution Practice) to remain between 2 °C and 8 °C. A nine-minute excursion to 9.4 °C during a routing delay needs adjudication: was it a real loss, was the cooling system at fault, who pays? The decision is regulated and

audit-grade — it cannot live in a spreadsheet.

The agents.

- [iot-coldchain-monitor](#) (edge, runs on a small Python runtime in the truck's cabin computer). System prompt: monitor the rolling 30-minute temperature window; if any reading is out of band, build a packet (timestamp + readings + GPS location) and publish to [cloud/escalation/excursion](#). Tools: [windowed_state](#), [mqtt_publish](#).
- [iot-excursion-adjudicator](#) (cloud). System prompt: given an excursion packet, determine if a partial-loss claim should be filed. Tools: [kb_search](#) against the GDP SOP collection, [connector_call](#) against the Sensitech telematics API for cooling-system telemetry, [approval_gate](#) requiring two signoffs from licensed pharmacists.
- [iot-claims-dispatcher](#) (cloud). System prompt: given an approved excursion adjudication, file a claim with the broker portal. Tools: [connector_call](#) to the broker portal.

The flow.

```
T+0      Sensor reading 9.4°C published to cold-chain/truck-42/temperature
      |
      ▼ (subscribed)
T+5ms    Edge monitor agent loads reading into windowed_state, sees
      30-min average above threshold → builds packet
      |
      ▼ publish cloud/escalation/excursion
T+800ms  Cloud adjudicator agent wakes
      |
      ├── kb_search("GDP cold-chain excursion thresholds")
      |   returns: 2.1.4 – excursions ≤15min ≤9.5°C are recoverable
      |
      ├── connector_call(sensitech, get_cooling_telemetry,
      |   truck=42, window=last_2h)
      |   returns: cooling system fault at T-12min
      |
      ├── reasoning: thresholds breached + fault confirmed
      |   → recommend partial-loss claim, $74,200
      |
      ▼ approval_gate(required_signoffs=2, expires_seconds=1800)
T+5s     Two pharmacists notified via Slack + email
      |
T+22m    Both pharmacists sign off in /approvals page
      |
      ▼ approved → triggers claims dispatcher
T+22m+3s connector_call(broker_portal, file_partial_loss_claim, ...)
      |
      ▼
T+22m+8s Claim CL-2026-0413 filed.
      Result published to cold-chain/truck-42/claim-id.
      Edge monitor agent stores the claim id in windowed_state
      so subsequent readings on the same shipment don't re-fire.
```

Why this couldn't be a single edge agent. The decision needs the GDP knowledge graph (~80 MB), a multi-step connector call to a third party, and two regulated humans. None of that fits on the truck. But the *trigger* — detecting the excursion in the first place — has to happen on the truck, because the GSM link drops when the truck is in mountain passes.

Why this couldn't be a single cloud agent. If the truck is offline, the adjudicator never sees the excursion. The split — edge does detection, cloud does adjudication — makes the system robust to network drops.

Scenario 3 — Beverage bottling alarm triage

The plant. A high-speed beverage bottling line, ~60,000 bottles per hour, 24x7, three shifts. The control system fires roughly 800 alarms a shift; an experienced operator can ignore 95% of them. New operators get overwhelmed and either stop the line for non-issues or miss the real ones.

The problem. Alarm fatigue. Real failures hide in the noise. A regulator audit finding from last quarter said "operator response inconsistent across shifts."

The agents.

- `iot-bedrocc-noise-filter` (edge, runs on a C runtime on a Phoenix Contact PLCnext gateway with 512 MB RAM). System prompt: classify each alarm as `noise` / `low` / `med` / `high` / `critical` based on a small rule set + recent context. Tools: `windowed_state`, `mqtt_publish`. Uses `code_executor` to run a 30-line Python pattern matcher.
- `iot-bedrocc-alarm-classifier` (cloud). System prompt: deeper LLM reasoning on `med+` alarms; cross-reference recent line state, last shift handover notes, and the maintenance log. Tools: `kb_search`, `tsdb_query`, `connector_call` to the maintenance log.
- `iot-bedrocc-safe-reset-advisor` (cloud). System prompt: when an operator wants to reset an alarm, run a 4-stage safety gate (hard interlocks -> authority matrix -> context preconditions -> minimum-privilege command) and produce an explicit allow/deny with rationale. Tools: `kb_search`, `approval_gate` (two ops sign-offs for any reset).

The flow.

```
Alarm fires on PLC
|
▼ PLC publishes to alarms/realtime
Edge noise-filter agent (sees ~800/shift)
├ "noise" → drop, increment counter, no publish.
├ "low"  → log to tsdb, dim banner on operator screen.
└ "med+" → publish to cloud/escalation/alarm-classify
    ▼
    Cloud classifier agent
    ├── kb_search("alarm code 0x4F2A")
    ├── tsdb_query(line state last 10min)
    ├── connector_call(maint log, last fault for asset)
    └ produces structured triage:
      { severity, root_cause_hypothesis, recommended_action }
    ▼
    Operator screen shows triage card
    |
    └ Operator: "I want to reset"
        ▼ POST /agents/iot-bedrocc-safe-reset-advisor/execute
```

```

Cloud safe-reset advisor
├─ stage 1: hard interlocks (vacuum chamber pressure < 50 mbar?)
├─ stage 2: authority (operator role allows reset of this class?)
├─ stage 3: context (no related alarm fired in last 30 min?)
├─ stage 4: minimum-privilege command (reset 0x4F2A, no broader)
└─ approval_gate(required_signoffs=2)
  ▼
Two ops sign off → command published to plant/line-A/reset-cmd
  │
  ▼ (subscribed)
PLC bridge issues the reset

```

The win. New operators get the same triage as veterans. Bad resets are blocked by the safety gate, not by tribal knowledge. Every reset has two signatures and a structured rationale that the auditor can replay.

Scenario 4 — Wind farm field maintenance

The plant. Same offshore wind farm. A field technician boards a turbine to investigate an alert. She has a tablet with spotty 4G, a phone, and her toolbox. Vendor manuals are PDFs in a SharePoint she can't reach offline.

The problem. She needs the OEM-cited repair procedure for fault code [E-1428: gearbox oil temperature high](#). She wants the exact torque values, the safety lockouts, and the parts list. She wants to dictate her closeout into a structured work order, not type it on a wet tablet.

The agents.

- [iot-fieldedge-troubleshoot-assistant](#) (cloud, but with a small offline-capable shadow on her tablet). System prompt: given a fault code and an asset, return the OEM procedure, the safety lockouts, the torque table, and a list of similar past WOs. Tools: [kb_search](#) against [rwe-fieldedge-oem-manuals](#), [image_analyzer](#) for damage photos, [connector_call](#) to the CMMS for past WOs.
- [iot-fieldedge-closeout-documenter](#) (cloud). System prompt: convert the technician's free-text or voice closeout into a structured WO. Tools: [audio_stt](#), [connector_call](#) to write the WO back.
- [iot-fieldedge-schedule-optimizer](#) (cloud). System prompt: given the day's backlog, technician availability, weather forecast, and ferry schedule, propose a 7-day schedule. Tools: [code_executor](#) (calls OR-tools), [connector_call](#) to weather, HRIS, ferry portal.

The flow.

```

Tech opens app, types "E-1428"
  ▼
App POSTs /agents/iot-fieldedge-troubleshoot-assistant/execute
  ▼
Cloud agent:
  - kb_search("E-1428 OEM manual")
    → "GE Haliade-X gearbox oil temp high"
  - kb_search("E-1428 safety procedure")
    → IEC 61400-1 + GE-specific lockout/tagout

```

```

- connector_call(SAP, search_past_wos, "E-1428 turbine X42")
  → 3 prior closeouts found
- synthesises: procedure + safety + parts + similar closures
▼
Tech reads procedure, performs repair, taps mic:
"replaced oil cooler element, torque 48 Nm,
  ran 30-min recirculation, no leaks, all clear"
▼
App POSTs /agents/iot-fieldedge-closeout-documenter/execute
  with audio file
▼
Cloud agent:
- audio_stt(audio) → text
- structured extraction: parts replaced, torque, tests done
- connector_call(SAP, update_wo, ...) → WO closed
▼
Done. Tech moves to the next turbine.

```

The edge angle. When her 4G drops mid-procedure, a small edge shadow on her tablet — a cut-down Python runtime — keeps a recently-cached version of the procedure and accepts her voice closeout. When the link comes back, the closeout flows up and the WO is updated. No "lost productivity" tickets because the agent kept working offline.

Scenario 5 — Distribution substation safe-reset

The plant. A medium-voltage distribution substation. A protection relay tripped on a phase imbalance. The operator at the control room wants to reset and re-energise. There are about 800 substations on this utility's network and the control room has 6 operators.

The problem. Resetting a relay without confirming the upstream cause is exactly how you get a transformer fire. The control room SOP is 14 pages. Different operators apply it differently. Audit findings cite "inconsistent re-energisation procedures."

The agents.

- [iot-substation-anomaly-classifier](#) (edge, runs on a Rust runtime in the substation cabinet). Watches relay events, classifies them into [transient](#) / [persistent](#) / [fault](#). Tools: [windowed_state](#), [mqtt_publish](#).
- [iot-substation-safe-reset-advisor](#) (cloud). 4-stage safety gate, identical pattern to Scenario 3 but with substation-specific rules. Tools: [kb_search](#) over the utility's protection-coordination KB, [tsdb_query](#) over upstream load history, [approval_gate](#) requiring a senior operator + a relay protection engineer.

The flow (compressed): edge classifier sees the trip, publishes the event to [cloud/escalation/substation-trip](#). Cloud agent runs the 4-stage gate. Two-signoff approval goes to the senior op + the protection engineer. On approval, the cloud agent publishes a reset command on [substation/sub-7/reset-cmd](#), the edge subscriber forwards to the relay, the relay re-energises. Total wall-clock: ~3 minutes (mostly the human signoff). Total handles touched by humans: zero.

The auditor's view. Same [/executions](#) page as everything else. Every reset has the four gate stages, the two human signatures, the cited KB clauses, the upstream load chart. Replayable.

Part VI — How an application uses Abenix

So far we've talked about agents that are triggered by sensors or by the platform UI. The third trigger is **another application** — your software calling Abenix as a service.

The SDK pattern

Abenix ships three SDKs out of the box: Python, TypeScript, Java. Same wire format, same mental model.

```
# Python
from abenix_sdk import Abenix, ActingSubject
forge = Abenix(
    base_url="https://abenix.your-corp.com",
    api_key="af_...", # platform-issued
    act_as=ActingSubject("contractiq", user_id, email, name), # NEW: end-user identity
)
result = forge.execute("contractiq-extract-clauses",
    {"contract_id": cid, "policy_kb": "msa-2026"})
print(result.output) # structured response from the agent
```

```
// Java — same pattern, stdlib HttpClient + Jackson, no glue.
try (Abenix forge = Abenix.builder()
    .baseUrl(System.getenv("ABENIX_API_URL"))
    .apiKey(System.getenv("CONTRACTIQ_ABENIX_API_KEY"))
    .actAs(new ActingSubject("contractiq", userId, email, name))
    .build()) {
    ExecutionResult res = forge.execute("contractiq-extract-clauses",
        Map.of("contract_id", cid, "policy_kb", "msa-2026"));
    System.out.println(res.output());
}
```

Three things to notice.

1. The application holds one API key

A SaaS app — say `contractiq` — gets one platform API key. It does NOT mint a key per end-user. That would be operationally absurd.

2. The application says *who* the work is for

`actAs` is the killer feature. The application passes the end-user's identity through to Abenix on every request. Internally Abenix routes that identity into:

- **Quotas** — the end-user's monthly token cap is what's checked, not the application's.
- **Audit log** — the row says "user `alice@bigco.com` ran `contractiq-extract-clauses` at 14:32 UTC", not "the contractiq service ran it".

- **Tenant isolation** — the end-user's own tenant scope is what reads/writes happen against. Two different end-users on the same SaaS see two different worlds.
- **RBAC** — the end-user's role + permissions are enforced. An end-user without the `extract-clauses` permission gets a 403, even though the application's key is fine.

This is what makes Abenix viable as a **white-label backend** for a SaaS product. The five showcase apps in this repo all use this pattern.

3. Same mental model as a function call

`forge.execute(slug, input)` — that's it. The agent could be one model call or a 9-step pipeline; could run in the cloud or proxy down to the edge. The application doesn't need to care.

Sync vs streaming vs async

Three modes:

Mode	When to use	How
Sync	Quick agents (<60 s). Simplest.	<code>forge.execute(slug, input)</code> — blocks until done, returns the final result.
Streaming	Long agents where the user wants to see progress.	<code>forge.stream(slug, input)</code> — returns an async iterator of <code>StreamEvent</code> (token, tool-call, dag-step) over Server-Sent Events.
Async	Fire-and-forget; check later.	<code>forge.execute(slug, input, wait=False)</code> — returns an <code>execution_id</code> immediately. Poll <code>/executions/{id}</code> or subscribe to the SSE for that id.

ClaimsIQ uses streaming for its live-DAG view. ContractIQ uses sync for quick clause extraction. The IoT live-mode toggle uses async because the run is a 3-minute pipeline.

Idempotency — the safety net

If your application is calling Abenix from a workflow that might retry on failure (cron, a queue worker, a third-party webhook), pass an `Idempotency-Key`:

```
result = forge.execute(slug, input, idempotency_key=f"shipment-{shipment_id}")
```

For 24 hours after the first call, any subsequent call with the same key returns the same cached result without re-running. Perfect for "the network blipped, did my filing actually go through?" scenarios.

Subscribing to outputs

For event-driven applications, you don't always want to call Abenix — you want Abenix to call you. Two patterns:

- **Webhook** — register a URL on the agent's manifest; Abenix posts the result to your URL.
- **MQTT** — subscribe to `executions/{slug}/output`; Abenix publishes every completed execution.

The cold-chain claims dispatcher works this way: it subscribes to `executions/iot-excursion-adjudicator/output` and fires the dispatch action whenever an adjudication completes.

Part VII — Anatomy of one execution

Walking line-by-line through one execution of `iot-pump-edge-classifier` so you see exactly what happens. (You'll never need to read this code yourself; it's here to show there's no magic.)

```
# 1. Caller sends a window
POST /agents/iot-pump-edge-classifier/execute
Body: {"samples": [0.012, -0.014, ...], "sample_rate_hz": 4096}
# 2. The runtime loads the manifest
slug = "iot-pump-edge-classifier"
manifest = yaml.safe_load(open(f"/var/edge/agents/{slug}/agent.yaml"))
system_prompt = open(f"/var/edge/agents/{slug}/system_prompt.md").read()
# 3. Construct the LLM call
messages = [{"role": "user", "content": json.dumps(input_body)}]
tools = [
    {"name": "code_executor",
     "description": "run a python snippet and return stdout",
     "input_schema": {"type": "object", "properties": {"code": {"type": "string"}}}},
    {"name": "mqtt_publish",
     "description": "publish to an MQTT topic from the allowed set",
     "input_schema": {"type": "object",
                      "properties": {"topic": {"type": "string"},
                                     "payload": {"type": "string"}}}},
]
# 4. The agent loop
response = anthropic.messages.create(
    model=manifest["model"],
    system=system_prompt,
    messages=messages,
    tools=tools,
    max_tokens=manifest["max_tokens"],
)
# 5. The model wants a tool — code_executor
# response.content[0] = {"type": "tool_use", "name": "code_executor",
#                        "input": {"code": "import numpy as np; ..."}}
# 6. Runtime executes the snippet in a subprocess
result = subprocess.run(
    ["python3", "-c", response.content[0].input["code"]],
    capture_output=True, timeout=5,
)
tool_output = result.stdout.decode()
# tool_output = '{"rms": 0.61, "peak_hz": 102.3}'
# 7. Feed back to the model
messages.append({"role": "assistant", "content": response.content})
messages.append({
    "role": "user",
    "content": [{"type": "tool_result",
                  "tool_use_id": response.content[0].id,
                  "content": tool_output}],
})
response = anthropic.messages.create(...same...)
# 8. The model now wants mqtt_publish
# response.content[0] = {"type": "tool_use", "name": "mqtt_publish",
#                        "input": {"topic": "plant-3/pump/p-12/severity",
#                                  "payload": '{"severity": "high", ...}'}}
# 9. Runtime checks the topic against the agent's MQTT publish ACL
allowed = manifest["edge_constraints"]["mqtt_publish"]
# allowed = ["plant-3/pump/+severity"]
# matches → permitted
```

```

mqtt_client.publish("plant-3/pump/p-12/severity",
                    response.content[0].input["payload"], qos=1)
# 10. Feed back, model returns a final text response
messages.append({"role": "assistant", "content": response.content})
messages.append({"role": "user",
                 "content": [{"type": "tool_result", "content": "ok"}]})
response = anthropic.messages.create(...)
# response.content[0] = {"type": "text",
#                        "text": '{"severity":"high","peak_hz":102.3,"rms_g":0.61}'}
# 11. Runtime returns the final result to the caller
return {
    "slug": slug,
    "duration_ms": int((time.time() - t0) * 1000),
    "result": json.loads(response.content[0].text),
}

```

That's the whole thing. The agent loop is at most 30 lines of code. The interesting work happens inside the model and inside the tools — the runtime is glue.

Part VIII — Security and governance

The whole edge story rests on three security primitives. They're worth understanding even if you're not the IT/OT lead.

Bundle signing

Every `.agent` bundle is signed with **RSA-PSS** (RSASSA-PSS, MGF1-SHA-256, salt-len 32) over the SHA-256 digest of the bundle bytes minus the signature itself. The signing key lives only in the platform; the public key is mounted into every gateway runtime at install time.

The runtime refuses to load a bundle whose signature doesn't verify. Period. There is no `--insecure` flag. An attacker who steals a bundle off the wire and modifies it cannot get it loaded; an attacker who steals the bundle and ships it to a *different* gateway *can* run it (if they also steal that gateway's `af_` token). That's the threat model.

The tool whitelist

Edge agents can only declare tools from this list:

```
mqtt_publish, mqtt_subscribe, current_time,
windowed_state, connector_call, code_executor
```

Specifically forbidden: `knowledge_search`, `kb_query`, `atlas_*`, `mcp_*`, `agent_step`, `approval_gate`, `human_approval`, `pipeline_*`. These either need cluster-only state (KB, Atlas, MCP), or require a human signoff (approvals), or call back to the platform (sub-pipelines). They have no business on a gateway.

The whitelist is enforced **twice**: once by the compiler (refuses to build a bundle that violates), once by the runtime (refuses to load even a correctly-signed bundle that violates).

MQTT ACLs per agent

Even within the allowed tools, agents are constrained by topic ACLs. An agent's manifest declares:

```
edge_constraints:
  mqtt_publish:
    - plant-3/pump/+/severity
    - cloud/escalation/bearing-anomaly
  mqtt_subscribe:
    - pump/vibration.raw
```

The runtime enforces these at every `mqtt_publish` call. An agent that's compromised at runtime — say, the model is jailbroken into calling `mqtt_publish("plant/critical/shutdown",`

"...") — has the publish dropped because the topic isn't in its ACL. Belt and braces against prompt injection.

Audit trail

Every execution — cloud or edge — emits a structured log line that flows into the platform's audit trail. The line includes the agent slug, the bundle digest, the gateway id (if edge), the caller identity (via `actAs`), the tool calls made, the topics published to, the duration, the failure code (if any). Tenant-scoped, integrity-hashed.

This means every reset of every relay, every claim filed, every work order opened, has a full chain of custody from "human request" through "agent reasoning" to "physical action." Replayable in [/executions](#). Searchable by `failure_code`, by `tenant`, by `gateway`.

Authentication

Channel	Auth mechanism
Web UI -> API	JWT (RS256), refresh-token flow, MFA optional
SDK (application) -> API	API key (<code>af_</code>), SHA-256-hashed at rest, scoped permissions, optional <code>actAs</code> for delegation
Edge runtime -> API	Same <code>af_</code> API key, sent as <code>Authorization: Bearer af_...</code>
MQTT broker	Configurable — anonymous in dev, mTLS or username+password in prod (passed via helm values)
Bundle integrity	RSA-PSS signature on the bundle itself, independent of MQTT auth

Defence in depth: a compromised MQTT broker still can't ship malicious bundles because the runtime checks the signature.

FAQ

Q: Do I need MQTT in my plant to use Abenix? A: No, but you'll get more out of the platform if you have it. The cloud half works without MQTT — pure HTTP. The edge half requires MQTT for OTA bundle delivery and async invocation. You can ship the same mosquitto we ship in our helm chart; it boots in 30 seconds and runs in 30 MB.

Q: Can I run an edge agent without the cloud? A: For short windows, yes — a deployed bundle keeps running until the next OTA push. The runtime queues telemetry and re-syncs when the link returns. For days-or-longer offline, there's a Phase-2 plan to bundle a distilled small model so the edge doesn't need to call out to Anthropic at all. Today the edge calls Anthropic over HTTPS through whatever uplink is available.

Q: How big is the edge runtime, really? A: Python ~80 MB, Rust ~25 MB, C ~12 MB. We optimise hard. The C runtime fits on a Phoenix Contact PLCnext with 256 MB of RAM and a 1 GHz Cortex-A7.

Q: Can two agents run on the same gateway? A: Yes. A gateway hosts an unbounded number of agents — they each get their own subdirectory under `/var/edge/agents/<slug>/` and their own slug-named topic subscription. Memory is the only practical limit.

Q: What if I need a tool that's not on the edge whitelist? A: The right answer is almost always: split the agent. Let the edge agent do the local-decision part, escalate to a cloud agent that has the missing tool. The cooperation pattern in Part IV covers this.

Q: How do I update an agent that's already deployed to 50 gateways? A: Click Deploy with the new revision. The platform fans out 50 MQTT publishes (one per gateway-deploy-topic). Each runtime hot-loads the new bundle. Total wall-clock for a 50-gateway fleet: typically under 30 seconds.

Q: How do I roll back? A: Each bundle has a digest; the gateway keeps the last 3 versions on disk. The platform UI has a Rollback button per gateway-agent pair that re-publishes the prior bundle.

Q: How does this compare to AWS Greengrass or Azure IoT Edge? A: Greengrass and IoT Edge are *device fleet management* platforms — they ship Lambda functions or container images. Abenix is an *AI agent* platform — it ships signed agent definitions that include the prompt, model, and tool wiring. You can run Abenix's edge runtime *on top of* Greengrass or IoT Edge if you want their device management, the same way you can run our Helm chart on AKS or EKS.

Q: Is the edge runtime open source? A: Yes. MIT licensed, same as the platform. Source for all three variants is in this repository under `apps/edge-runtime{,-rust,-c}/`.

Q: How do I monitor the fleet? A: The platform's [/edge](#) page shows every registered gateway with last-seen-at and deployed-bundle digests. Prometheus metrics ([abenix_edge_gateway_last_seen_seconds](#), [abenix_edge_bundle_load_total](#), [abenix_edge_execution_duration_seconds](#)) are scraped from each runtime. Grafana dashboards ship in [infra/helm/abenix/templates/grafana-dashboards/](#).

Glossary

- **Agent** — a small specialised program with a name, a prompt, a model, and a list of tools. The unit of deployment in Abenix.
- **Tool** — a named function the agent can call (read sensor, write to PLC, search KB, etc.). 100+ ship in the box.
- **Pipeline** — a DAG of agents and tools. Switch nodes branch; loop nodes iterate.
- **Bundle (.agent)** — a signed tarball containing one agent's manifest, prompt, and inline tools. The OTA payload to the edge.
- **Gateway** — a runtime pod sitting on the plant LAN, hosting deployed bundles. Python / Rust / C variants.
- **Edge constraint** — per-agent declarations: max payload, max runtime, allowed MQTT topics. Enforced at compile and load.
- **Topic** — an MQTT address; forward-slash hierarchical strings like [pump/vibration.raw](#).
- **QoS** — MQTT delivery guarantee: 0 (fire-and-forget), 1 (at least once), 2 (exactly once).
- **Retained message** — an MQTT message the broker keeps as the "current state" for late subscribers.
- **actAs** — the Abenix delegation pattern; an application holds one platform key and passes per-end-user identity on each call.
- **Idempotency-Key** — an HTTP header that makes a retried [/execute](#) return the cached previous result for 24 h.
- **Approval gate** — a pipeline node that blocks execution until N humans sign off, with TTL.
- **Failure code** — a stable string identifier for a class of failure ([LLM_RATE_LIMIT](#), [SANDBOX_TIMEOUT](#), etc.). The [/alerts](#) page groups by it.
- **Atlas** — the unified ontology + KB canvas. One graph for the document corpus and the typed concepts.
- **DLQ** — Dead-Letter Queue. Failed executions land here with one-click replay or discard.

This document was written to be shared. If you've read this far and you're considering Abenix for your operations, get in touch with us — the showcase apps in the repository are open and runnable, and the team behind the platform is happy to walk a real plant through the first deployment.